

System Design Basics

Lesson 4: Caching Strategies

Caching Strategies - Study Notes

Key Concepts

1. **Client Side Caching** – Storing responses in the browser or device to avoid repeated network calls.
2. **CDN (Content Delivery Network) Caching** – Edge servers replicate static assets close to users, reducing latency and origin load.
3. **Database (Server Side) Caching** – In memory stores (e.g., Redis, Memcached) keep frequently accessed query results or objects to spare the DB.
4. **Cache Invalidation & Consistency** – Strategies for keeping cached data fresh (TTL, write through, write behind, cache aside).
5. **Cache Hierarchy** – Combining client, CDN, and DB caches to achieve optimal performance and scalability.

Summary

Caching is a fundamental technique for improving the speed and scalability of web applications. By keeping copies of data closer to where it is needed, we reduce round trip time, lower bandwidth consumption, and relieve pressure on backend services.

Three primary layers are typically employed:

- **Client side caching** leverages HTTP headers (`Cache-Control`, `ETag`, `Expires`) and browser storage APIs (LocalStorage, IndexedDB) so that repeat visits can reuse resources without contacting the server.
- **CDN caching** places static files (images, CSS, JavaScript, even pre-rendered HTML) on a global network of edge nodes. When a user requests a resource, the CDN serves it from the nearest node, dramatically cutting latency and off-loading the origin server.
- **Database caching** uses fast, in-memory data stores such as Redis. Frequently requested data—like product catalogs, user sessions, or computed aggregates—are read from Redis instead of hitting the relational database, cutting query time from milliseconds to microseconds.

A practical example is caching product data in Redis. When the application needs a product's details, it first checks Redis. If the entry exists (a **cache hit**), the data is returned instantly. If not (a **cache miss**), the app queries the primary DB, returns the result to the client, and writes the product object into Redis for future requests. Proper invalidation (e.g., setting a TTL of 5 /minutes or purging the key on updates) ensures the cache stays consistent with the source of truth.

Important Points

- **Cache-Control Headers** (`public`, `private`, `max-age`, `must-revalidate`) dictate how browsers and CDNs store and reuse responses.
- **TTL (Time to Live)** defines how long an entry lives in a cache before it is considered stale.

- **Cache Invalidation Patterns**
 - **Write through**: write to cache and DB simultaneously.
 - **Write behind (write back)**: write to cache first, propagate to DB asynchronously.
 - **Cache aside (lazy loading)**: application reads/writes DB, manually populates/evicts cache.
 - **Redis Data Types** (strings, hashes, sets, sorted sets, streams) let you model many caching scenarios efficiently.
 - **Cold Cache vs. Warm Cache** – The first request after a restart may be slower; warm up scripts can pre populate critical keys.
 - **Cache Stampede** – When many requests miss simultaneously; mitigations include request coalescing, probabilistic early expiration, or “lock step” refresh.
 - **Monitoring** – Track hit rate, miss rate, latency, and eviction metrics to gauge cache effectiveness.

Examples

1. Redis Cache for Product Data (Node.js/Express)

```
```js
const redis = require('redis');
const client = redis.createClient(); // default localhost:6379
const db = require('./db'); // hypothetical DB module
app.get('/products/:id', async (req, res) => {
 const key = `product:${req.params.id}`;
 // 1p ã Try cache first
 const cached = await client.get(key);
 if (cached) {
 return res.json(JSON.parse(cached)); // cache hit
 }
 // 2p ã Cache miss – fetch from DB
 const product = await db.getProductById(req.params.id);
 if (!product) return res.status(404).send('Not found');
 // 3p ã Store in Redis with TTL (e.g., 300 seconds)
 client.setex(key, 300, JSON.stringify(product));
 res.json(product);
});
```
```

***Explanation*:**

- ``GET /products/:id`` first checks Redis (``client.get``).
- On a miss, it queries the relational DB, then writes the result back to Redis with ``SETEX`` (set + expiration).
- Subsequent requests within 5 /minutes hit the cache, returning data almost instantly.

2. Browser Cache for Static Assets (HTML Header)

```
```http
Cache-Control: public, max-age=86400, immutable
```

---

**\*Explanation\*:**

- ``public`` allows any cache (browser, CDN) to store the response.
- ``max-age=86400`` (24 /h) tells caches the resource is fresh for a day.
- ``immutable`` signals that the file will never change, so the browser can skip revalidation entirely.
- Ideal for versioned assets like ``style.v1.2.3.css``; when the file changes, the URL changes, avoiding stale content.

---

## Quick Review

1. **\*\*What are the three main layers of caching discussed, and where does each reside?\*\***
2. **\*\*Name two cache invalidation strategies and briefly describe when you might use each.\*\***
3. **\*\*In the Redis example, what purpose does ``SETEX`` serve?\*\***
4. **\*\*How does the ``Cache-Control: immutable`` directive affect browser behavior?\*\***
5. **\*\*What is a “cache stampede,” and one technique to prevent it?\*\***

---

## Further Reading

- **\*\*Advanced Redis Patterns\*\*** – Lua scripting, Redis Streams, and geo spatial caching.
- **\*\*CDN Configuration\*\*** – Edge rules, cache key customization, and origin pull vs. push.
- **\*\*Cache Invalidation at Scale\*\*** – Using message queues (Kafka, RabbitMQ) to propagate updates.
- **\*\*HTTP Caching Deep Dive\*\*** – ETag, Last Modified, Vary, and conditional requests.
- **\*\*Observability\*\*** – Prometheus metrics for cache hit/miss ratios, Grafana dashboards, and alerting.

